

1. State the following functions: Partial, Constant and Total. (3 M)

(a) Partial Function

A **partial function** is a function that is **not defined for every input**.

- For some inputs, the output may not exist.
- Example:

$$f(x) = \frac{1}{x}$$

This is not defined when $x = 0$.

(b) Constant Function

A **constant function** always gives the **same output** for every input.

- Example:

$$f(x) = 5$$

Whatever value of x , the answer is always 5.

(c) Total Function

A **total function** is defined for **all possible inputs**.

- Every input has an output.
- Example:

$$f(x) = x + 2$$

It works for every integer x .

2. Primitive Recursive Functions. (3M, 4M, 7M)

Primitive Recursive Functions are a special class of computable functions that are **always defined** (total functions) and **always terminate** (never enter infinite loops). They form a fundamental subset of all computable functions in theoretical computer science.

Key Characteristics

Feature	Description
Always terminates	No infinite loops possible
Total function	Defined for all possible inputs
Built from basics	Zero, Successor, and Projection functions
Two operations	Composition and Primitive Recursion
Subset of computable	Weaker than μ -recursive functions builds.openlogicproje +1

Primitive recursive functions are constructed from three **initial functions**:

1. **Zero Function:** $Z(x) = 0$
2. **Successor Function:** $S(x) = x + 1$
3. **Projection Function:** $P_i^k(x_1, \dots, x_k) = x_i$ (selects the i-th input) [tutorialspoir +1](#)

Important Limitations

- **Cannot express all computable functions** (weaker than μ -recursive functions)
- **Cannot express Ackermann's function** (requires unbounded search)
- **Guaranteed to terminate** (unlike general recursive functions) [wikipedia](#) [youtube](#)

What is a Projection Function?

Think of it as a "selector" — it simply **picks one input** from a list of inputs and returns it.

Simple Definition:

$$P_i^k(x_1, x_2, x_3, \dots, x_k) = x_i$$

Translation: "From the list of k inputs, return the i-th input."

Example 1: 2 inputs (k = 2)

Function	Picks	Example
$P_1^2(x, y) = x$	Returns first input	$P_1^2(5, 10) = 5$
$P_2^2(x, y) = y$	Returns second input	$P_2^2(5, 10) = 10$

Example 2: 3 inputs (k = 3)

Function	Picks	Example
$P_1^3(x, y, z) = x$	Returns first input	$P_1^3(2, 7, 9) = 2$
$P_2^3(x, y, z) = y$	Returns second input	$P_2^3(2, 7, 9) = 7$
$P_3^3(x, y, z) = z$	Returns third input	$P_3^3(2, 7, 9) = 9$

Example: Addition

text

```
add(x, 0) = x

add(x, y+1) = add(x, y) + 1
```

Step-by-Step Calculation: add(2, 3)

Let's compute 2 + 3:

text

```
add(2, 3)
= add(2, 2) + 1           (apply recursive step)
= [add(2, 1) + 1] + 1     (apply recursive step again)
= [[add(2, 0) + 1] + 1] + 1 (again)
= [[2 + 1] + 1] + 1       (base case: add(2, 0) = 2!)
= [3 + 1] + 1
= 4 + 1
= 5 ✓
```

Result: 2 + 3 = 5

Example: Multiplication

text



```
mult(x, 0) = 0                                ← BASE CASE
mult(x, y + 1) = add(x, mult(x, y))          ← RECURSIVE STEP
```

Step-by-Step: mult(3, 2)

Let's compute 3×2 :

text



```
mult(3, 2)
= add(3, mult(3, 1))          (recursive step)
= add(3, add(3, mult(3, 0)))  (recursive step again)
= add(3, add(3, 0))          (base case: mult(3, 0) = 0)
= add(3, 3)                  (add(3, 0) = 3 from base case)
= 6                          (we know add(3, 3) = 6) ✓
```

Result: $3 \times 2 = 6$

3. Describe Recursive function. Prove every recursive function is computable.

A **recursive function** is a function that **calls itself** to solve a problem. It breaks a big problem into smaller, similar problems until it reaches a simple case that it can solve directly.

Two Essential Parts of Every Recursive Function

Part	Name	What it does	Why it's needed
1	Base Case	Simplest input with direct answer	Stops recursion (prevents infinite loop) sciencedirect
2	Recursive Step	Function calls itself with smaller input	Solves the problem by breaking it down sciencedirect

Simple Example: Factorial

The factorial function is the **classic recursive function**:

text

```
factorial(n) =  
    1          if n = 0      ← BASE CASE  
    n × factorial(n-1)  if n > 0 ← RECURSIVE STEP
```

Mathematical notation:

$$f(n) = \begin{cases} 1 & \text{if } n = 0 \\ n \times f(n-1) & \text{if } n > 0 \end{cases}$$

Step-by-Step: Calculate factorial(4)

text

```
factorial(4)  
= 4 × factorial(3)          (recursive step)  
= 4 × (3 × factorial(2))    (recursive step)  
= 4 × (3 × (2 × factorial(1))) (recursive step)  
= 4 × (3 × (2 × (1 × factorial(0)))) (recursive step)  
= 4 × (3 × (2 × (1 × 1)))    (BASE CASE: factorial(0) = 1!)  
= 4 × 3 × 2 × 1 × 1  
= 24 ✓
```

Proof That Every Recursive Function is Computable

What Does "Computable" Mean?

A function is computable if there exists an algorithm (or Turing Machine) that can calculate it for any valid input.

Take factorial:

- $fact(0) = 1$
- $fact(n) = n \times fact(n-1)$

To compute $fact(4)$:

- $fact(4) = 4 \times fact(3)$
- $fact(3) = 3 \times fact(2)$
- $fact(2) = 2 \times fact(1)$
- $fact(1) = 1 \times fact(0)$
- $fact(0) = 1$

So:

- $fact(4) = 24$

This shows recursion gives a clear algorithm, so it is computable.

A recursive function in computability theory is a function defined using formal rules like:

- zero function,
- successor function,
- projection function,
- composition,
- primitive recursion,
- sometimes minimization.

These rules are mechanical. That is why such a function can be computed by a step-by-step algorithm.

Why it is computable: To show a function is computable, we only need to show that a machine or algorithm can calculate it in finite steps.

Now look at the building blocks:

- Zero function: always gives 0.
- Successor function: adds 1.
- Projection function: just selects one input.

These are obviously computable.

Then:

- Composition means combining computable functions.
- Primitive recursion means defining a function using a base case and then repeatedly using smaller values.

If each small part is computable, then the whole function is computable.

Easy example: addition

Addition is defined recursively as:

- $add(x, 0) = x$
- $add(x, y + 1) = add(x, y) + 1$

To compute $add(2, 3)$, we do:

- $add(2, 0) = 2$
- $add(2, 1) = 3$
- $add(2, 2) = 4$
- $add(2, 3) = 5$

This is a **finite process**, so it is computable.

Recursive functions are obtained from the basic computable functions zero, successor, and projection by using composition and primitive recursion. Since the basic functions are computable, and composition and primitive recursion preserve computability, every recursive function can be calculated by an algorithm. Hence every recursive function is computable.

Difference between Primitive Recursive Function and Recursive Function

- **Primitive recursive functions** are a **smaller class** of recursive functions that always **terminate**.
- **Recursive functions** (also called **general recursive functions**) are a **larger class** that may use **unbounded minimization**, so they may be **partial** and may not always terminate.

Easy comparison

Point	Primitive Recursive	Recursive
Termination	Always terminates	May or may not terminate
Output	Always total	Can be partial
Construction	Zero, successor, projection, composition, primitive recursion	Same, plus minimization
Power	Less powerful	More powerful
Example	Addition, multiplication, factorial	Ackermann-like functions, search-based functions

4. Prove that for two recursive languages L_1 and L_2 their union and intersection is recursive. (7 M)

Let L_1 and L_2 be recursive languages. Then there exist Turing machines M_1 and M_2 such that:

$$L(M_1) = L_1, \quad L(M_2) = L_2$$

and both M_1 and M_2 halt on every input.

1. Union is recursive

We construct a Turing machine $M_{\cup}$ for $L_1 \cup L_2$ as follows:

$$M_{\cup}(w) = \begin{cases} \text{accept} & \text{if } M_1(w) \text{ accepts} \\ \text{accept} & \text{if } M_1(w) \text{ rejects and } M_2(w) \text{ accepts} \\ \text{reject} & \text{if } M_1(w) \text{ rejects and } M_2(w) \text{ rejects} \end{cases}$$

Since M_1 and M_2 both halt on every input, $M_{\cup}$ also halts on every input. Therefore,

$$L_1 \cup L_2 \text{ is recursive.}$$

Equivalently, for every string w ,

$$w \in L_1 \cup L_2 \iff (w \in L_1) \vee (w \in L_2)$$

2. Intersection is recursive

We construct a Turing machine $M_{\cap}$ for $L_1 \cap L_2$ as follows:

$$M_{\cap}(w) = \begin{cases} \text{accept} & \text{if } M_1(w) \text{ accepts and } M_2(w) \text{ accepts} \\ \text{reject} & \text{if } M_1(w) \text{ rejects} \\ \text{reject} & \text{if } M_1(w) \text{ accepts and } M_2(w) \text{ rejects} \end{cases}$$

Again, since M_1 and M_2 both halt on every input, $M_{\cap}$ also halts on every input. Hence,

$$L_1 \cap L_2 \text{ is recursive.}$$

Equivalently, for every string w ,

$$w \in L_1 \cap L_2 \iff (w \in L_1) \wedge (w \in L_2)$$

Let

$$L_1 = \{w \mid w \text{ represents an even number}\}$$

and

$$L_2 = \{w \mid w \text{ represents a number divisible by 3}\}$$

Both are **recursive languages** because we can decide them by a machine that always halts.

Union with example

The union is

$$L_1 \cup L_2 = \{w \mid w \in L_1 \text{ or } w \in L_2\}$$

So a string is in the union if it is **even or divisible by 3**.

Example

Take the number 6.

- $6 \in L_1$ because 6 is even.
- $6 \in L_2$ because 6 is divisible by 3.

So,

$$6 \in L_1 \cup L_2$$

Take the number 5.

- $5 \notin L_1$
- $5 \notin L_2$

So,

$$5 \notin L_1 \cup L_2$$

Intersection with example

The intersection is

$$L_1 \cap L_2 = \{w \mid w \in L_1 \text{ and } w \in L_2\}$$

So a string is in the intersection if it is **both even and divisible by 3**.

Example

Take the number 6.

- $6 \in L_1$
- $6 \in L_2$

So,

$$6 \in L_1 \cap L_2$$

Take the number 4.

- $4 \in L_1$
- $4 \notin L_2$

So,

$$4 \notin L_1 \cap L_2$$

5. Discuss μ -recursive functions (3 M)

These functions use something called the μ (mu) operator.

μ means:

👉 "Keep searching until condition becomes true."

Example

Find smallest y such that:

$$y + 3 = 7$$

Try values:

- $0 + 3 = 3$
- $1 + 3 = 4$
- $2 + 3 = 5$
- $3 + 3 = 6$
- $4 + 3 = 7$ ✓

Answer:

$$y = 4$$

3. Bounded Minimalization

This means:

👉 "Find the smallest value that satisfies a condition, but search only up to a limit."

Example

Find smallest number $y \leq 5$ such that:

$$y^2 = 4$$

Check one by one:

- $0^2 = 0$
- $1^2 = 1$
- $2^2 = 4$ ✓

Smallest answer is:

$$y = 2$$

We searched only till 5.

👉 This is called **Bounded Minimalization**.



In bounded minimalization:

- search has limit.

In μ -recursive:

- there may be no limit.

So sometimes search may continue forever.

6. Define: Bounded Minimalization and show that, if P is a primitive recursive (n + 1) place predicate, bounded minimalization m is a primitive recursive function. (7 M)

2. What is a Predicate?

A predicate is simply a condition that gives:

- TRUE or FALSE

Example:

$$P(x, y) : x + y = 5$$

This becomes TRUE for:

$$(2, 3), (1, 4), (5, 0)$$

and FALSE for many other values.

3. What Does "(n + 1)-place Predicate" Mean?

It means the predicate has:

- $n + 1$ variables.

Example:

$$P(x_1, x_2, \dots, x_n, y)$$

Here:

- $x_1, x_2, \dots, x_n$ are first n variables
- y is one more variable

So total variables:

$$n + 1$$

4. Definition of Bounded Minimalization

Bounded minimalization means:

👉 “Find the smallest value of y satisfying a condition, but search only up to a fixed limit.”

It is written as:

$$m(x_1, x_2, \dots, x_n, z) = \mu y \leq z [P(x_1, x_2, \dots, x_n, y)]$$

5. Easy Meaning of the Formula

This formula means:

- Check values:

$$y = 0, 1, 2, \dots, z$$

- Find the first (smallest) value of y for which predicate P becomes TRUE.
- If no value works up to z , then answer is usually:

$$z + 1$$

6. Simple Example

Suppose:

$$P(x, y) : x + y = 5$$

Take:

$$x = 2, \quad z = 5$$

We must find smallest $y \leq 5$ such that:

$$2 + y = 5$$

Check one by one:

y	$2 + y$	True/False
0	2	False
1	3	False
2	4	False
3	5	True 

First TRUE occurs at:

$$y = 3$$

Therefore:

$$m(2, 5) = 3$$

7. Now We Prove It is Primitive Recursive

We are given:

- 👉 Predicate P is primitive recursive.

Need to prove:

- 👉 Bounded minimalization function m is also primitive recursive.
-

8. Main Idea of the Proof

Primitive recursive functions:

- always finish,
- never run forever.

So we must show:

- bounded minimalization also finishes in finite steps.

9. Why Does the Search Always Finish?

In bounded minimalization we search only:

$$0, 1, 2, \dots, z$$

This is a finite list.

So:

- after checking at most $z + 1$ values,
- search definitely stops.

No infinite loop is possible.

10. Since P is Primitive Recursive

Given:

$$P(x_1, x_2, \dots, x_n, y)$$

is primitive recursive.

That means:

- checking whether P is TRUE or FALSE
is computable using finite steps.

So every test:

$$P(\dots, 0), P(\dots, 1), P(\dots, 2)$$

can be computed easily.



11. Finite Search Can Be Done by Primitive Recursion

We search:

$$0 \rightarrow z$$

step-by-step.

This is exactly the kind of repeated finite process allowed in primitive recursion.

So:

- finite checking,
- repeated testing,
- stopping at first TRUE

can all be built using primitive recursive operations.

12. Therefore

Because:

1. P is primitive recursive
2. Search range is finite
3. Finite search can be done using primitive recursion

the bounded minimalization function:

$$m(x_1, \dots, x_n, z)$$

is also primitive recursive.



7. Define and explain Bounded Quantification. (7M)

Bounded Quantification

Definition

Bounded quantification means:

A variable is checked only within a fixed limit (bounded range).

Instead of checking infinitely many values, we check only a finite number of values.

This makes computation easy and guaranteed to finish.

Types of Bounded Quantification

There are two types:

1. Bounded Universal Quantification
2. Bounded Existential Quantification

1. Bounded Universal Quantification

It is written as:

$$(\forall y \leq z) P(y)$$

Meaning

This means:

Predicate $P(y)$ is true for **all values** of y from 0 to z .

We check:

$$0, 1, 2, \dots, z$$

If all satisfy the condition, the statement is TRUE.

Example

Check:

$$(\forall y \leq 5)(y < 10)$$

We test values:

y	$y < 10$
0	True
1	True
2	True
3	True
4	True
5	True

Since all are true,

$$(\forall y \leq 5)(y < 10)$$

is TRUE.

2. Bounded Existential Quantification

It is written as:

$$(\exists y \leq z) P(y)$$

Meaning

This means:

There exists at least one value of y between 0 and z for which $P(y)$ is true.

Example

Check:

$$(\exists y \leq 5)(y^2 = 4)$$

Test values:

y	y^2
0	0
1	1
2	4 

At $y = 2$, condition becomes true.

Therefore:

$$(\exists y \leq 5)(y^2 = 4)$$

is TRUE.

Why is it Called "Bounded"?

Because the variable is restricted by a limit.

Example:

$$y \leq 5$$

So we check only:

$$0, 1, 2, 3, 4, 5$$

The checking is finite.

Difference Table

Point	Bounded Minimalization	Bounded Quantification
Meaning	Finds the smallest value y that satisfies a predicate, within a fixed bound z .	Checks whether a predicate is true for values up to a fixed bound z .
Main operation	Search and return one value.	Check truth of a statement.
Notation	$\mu y \leq z P(x_1, \dots, x_n, y)$	$\forall y \leq z P(y)$ or $\exists y \leq z P(y)$
Output	A number (the least satisfying value).	True/False.
Purpose	To locate the first satisfying value.	To test whether a condition holds for some or all values in a finite range.
Termination	Always terminates because the search is bounded.	Always terminates because only finitely many values are checked.
Use in theory	Helps define primitive recursive functions.	Helps define primitive recursive predicates.
Example	Smallest $y \leq 5$ such that y is even $\rightarrow y = 0$.	$\exists y \leq 5$ such that y is even $\rightarrow$ True.

Type	Easy meaning	Example
Primitive recursive	Fixed recursion, always stops	Addition
Recursive	General computable function	Factorial, addition
μ -recursive	Search for smallest answer	Least y such that $y^2 = 4$

Concept	Easy Meaning	Easy Mathematical Example
Recursive Function	Function solves problem by calling itself again and again	Sum of first n numbers: $S(0) = 0$ $S(n) = n + S(n - 1)$ Example: $S(3) = 3 + 2 + 1 + 0 = 6$
Primitive Recursive Function	Recursive function that always finishes	Multiplication: $mult(x, 0) = 0$ $mult(x, y + 1) = mult(x, y) + x$ Example: $mult(3, 2) = 6$
μ -Recursive Function	Keep checking values until condition becomes true	Find smallest y such that: $y^2 = 16$ Checking: 0, 1, 2, 3, 4 Since: $4^2 = 16$ Answer: $\mu y(y^2 = 16) = 4$
Bounded Minimalization	Search only within a fixed limit	Find smallest $y \leq 10$ such that: $y + 2 = 5$ Checking: 0 + 2, 1 + 2, 2 + 2, 3 + 2 Since: $3 + 2 = 5$ Answer: $\mu y \leq 10(y + 2 = 5) = 3$
Bounded Quantification	Check conditions only within a fixed range	Check whether all numbers up to 4 are less than 10: $(\forall y \leq 4)(y < 10)$ Also check whether some number up to 5 satisfies: $(\exists y \leq 5)(y + 1 = 4)$